

The Rise and Fall of Google’s Privacy Sandbox

Rachid Youssef Grib
Politecnico di Torino
Torino, Italy
rachidyoussef.grib@studenti.polito.it

Alberto Verna
Politecnico di Torino
Torino, Italy
alberto.verna@polito.it

Nikhil Jha
Politecnico di Torino
Torino, Italy
nikhil.jha@polito.it

Martino Trevisan
University of Trieste
Trieste, Italy
martino.trevisan@dia.units.it

Marco Mellia
Politecnico di Torino
Torino, Italy
marco.mellia@polito.it

A Artifact Appendix

A.1 Abstract

The artifact contains the code, Docker recipes, execution scripts, and processed datasets used to measure the deployment of Google Privacy Sandbox APIs on the Web. It packages a privacy banner-aware crawler, tools for extracting Privacy Sandbox attestation information, post-processing scripts, and open datasets with Jupyter notebooks that reproduce the figures in the paper. When executed, the artifact runs the measurement pipeline over a configurable Tranco top-site list and produces zip archives containing the final CSV outputs. The released code is distributed under the AGPLv3 licence. The artifact is available at [10.5281/zenodo.20759216](https://zenodo.org/doi/10.5281/zenodo.20759216).

A.2 Description & Requirements

The artifact root contains a `README.md` with the complete workflow. The artifacts are organised into the following directories:

- `crawler/`: a Selenium-based crawler that accepts or denies consent banners and records Privacy Sandbox usage;
- `extract-allowed/`: scripts for obtaining the list of domains that are allowed to invoke Privacy Sandbox APIs;
- `post-process/`: tools for extracting contacted domains, checking attestation files, and compacting crawler JSON into CSV;
- `open-data/`: the filtered datasets and notebooks used for the paper’s plots.

We also provide the following scripts:

- `install-dependencies.sh`: installs system dependencies on Ubuntu LTS;
- `build.sh`: builds the Docker images for the crawler and post-processing tools;
- `analyze-ps-complete.sh`: runs the full measurement campaign from Tranco list download to final CSV output, with configurable parameters;
- `analyze-ps-single.sh`: runs a single crawl of a specified website and produces a JSON summary of the results.

A.2.1 Security, privacy, and ethical concerns. The full workflow performs active Web crawls, downloads public Tranco and Privacy Sandbox attestation data, and contacts websites and third parties loaded by those websites. Therefore, our artifacts do not pose significant security risks to the host system. We however recommend running the crawler from a network where collecting such traffic is acceptable. Moreover, running a single instance of the crawler visits

single website twice in the top-10,000 list. We deem this a negligible volume of traffic, which we expect does not cause any problem to websites of such size. For the same reason, we recommend against running the crawler multiple times on the same website.

A.2.2 How to access. The artifacts are available on Zenodo at the following DOI: [10.5281/zenodo.20759216](https://doi.org/10.5281/zenodo.20759216). They can also be accessed from the following GitHub repository: <https://github.com/AI4CTI/priv-accept-ps>.

A.2.3 Hardware dependencies. No specialised hardware is required. A single modern x86-64 Linux machine is sufficient for functionality tests and small crawls. The full 10,000-website crawl is CPU-, network-, and memory-intensive. On our machine, that allowed ~20 parallel crawler instances, each top-10,000 cycle was completed in roughly 36 hours.

A.2.4 Software dependencies. The campaign scripts target Ubuntu LTS systems and were documented as tested on Ubuntu 22.04/24.04 LTS. They require Docker, GNU Parallel, zip, and unzip; `install-dependencies.sh` installs these packages on Ubuntu. This project uses Docker containers to encapsulate the crawler and post-processing tools. The images can be built locally with `build.sh` or pulled from Docker Hub (`salb98/<name>` where `<name>` is the image name as specified in `build.sh`). By default, the analysis script uses the local images, but it can be modified to pull from Docker Hub instead. The open-data notebooks require Python 3.13.x and install their Python dependencies automatically.

A.2.5 Benchmarks. The main benchmark is the weekly Tranco top-10,000 website crawl from 23 June 2025 to 5 January 2026, available from <https://tranco-list.eu/>. The artifact includes processed archives in `open-data/outputs-longitudinal/` and paired Chrome/Firefox snapshots in `open-data/outputs-firefox-vs-chrome/`. Each archive contains `allowed_domains.csv`, `attested_domains.csv`, `allowed_attested.csv`, and `crawler_outputs.csv`. The notebooks also use `domain_map.json`, derived from DuckDuckGo Tracker Radar, to map domains to owning entities.

A.3 Set Up

A.3.1 Installation. On an Ubuntu LTS host, the setup consists of installing the system dependencies, rebooting so Docker group membership takes effect, and building the artifact containers:

```
cd /path/to/artifacts
bash install-dependencies.sh
# reboot, then return to the artifact directory
```

```
bash build.sh
```

The build creates the Docker images used by the allowed-domain extraction, crawling, and post-processing stages. Evaluators who only wish to reproduce the paper figures from the released data can instead open the notebooks in `open-data/` with Python 3.13.x.

A.3.2 Basic test. After building the images, run a small functionality test over the first 10 websites of the Tranco list:

```
bash analyze-ps-complete.sh -w 10 -p 1
```

The `-w <N>` parameter limits the analysis to the top- N websites, while `-p <M>` sets the maximum number of parallel crawler instances to M .

A successful run downloads the Tranco list, extracts the allowed-domain list, performs accept and deny crawls for the top 10 websites, and executes the post-processing stages. When the process is done, the scripts outputs `Done!` on the console. In that case, the final outputs should be present at the path `outputs/outputs-YYYYMMDD.zip` from the working folder. This zip file should contain three CSV files: `attested_domains.csv`, `allowed_domains.csv`, `allowed_attested.csv`, and `crawler_outputs.csv`. Furthermore, the raw, unprocessed outputs are available in the `outputs-YYYYMMDD` folder.

A.4 Evaluation Workflow

A.4.1 Major claims.

- (C1): The crawler provides a longitudinal, consent-aware measurement of Privacy Sandbox API usage across the Web. This supports the methodology and dataset claims in Section 4 and Table 2.
- (C2): Privacy Sandbox adoption was limited and concentrated: only CHIPS, Topics, and Attribution Reporting exceeded 10% After-Accept website presence in the latest snapshot, while most APIs remained marginal. This is reported in Section 5 and Figures 1–3.
- (C3): CHIPS was the only broadly adopted and sustained Privacy Sandbox component, but in Chrome it coexisted with widespread traditional unpartitioned third-party cookies. This is reported in Section 6 and Figures 4–5.
- (C4): Retired APIs were already declining before Google’s retirement announcement, with Protected Audience and Shared Storage collapsing to near-zero usage and caller counts remaining small. This is reported in Section 7 and Figure 6.
- (C5): The maintained APIs other than CHIPS did not show meaningful growth: Private State Tokens, FedCM, and Fenced Frames remained low or absent. This is reported in Section 8 and Figures 7–8.
- (C6): Some Privacy Sandbox and cookie activity appears before consent or after denial, but the patterns suggest inconsistent CMP integration rather than direct evidence of deliberate bypassing. This is reported in Section 9 and Figures 9–11.

A.4.2 Experiments.

- (E1): **Reproduce longitudinal figures** [15 person-minutes + about 1 compute-hour + 32 GB RAM + less than 10 GB disk]. This validates C1, C2, C4, C5, and C6 using the released processed datasets.

Preparation: Open `open-data/longitudinal-plots.ipynb` in a Python 3.13.x Jupyter environment. Keep the zip archives in `open-data/outputs-longitudinal/` in place.

Execution: Run all notebook cells. The notebook installs dependencies, loads the archived CSVs, computes API usage and caller statistics, and stores plots under `open-data/plots-longitudinal/`.

Results: The generated plots reproduce the longitudinal and consent-state results in Figures 1–3 and Figures 5–9, including the decline of retired APIs, the stable dominance of CHIPS, and the small caller population for most APIs.

- (E2): **Reproduce Chrome/Firefox cookie comparison** [10 person-minutes + less than 1 compute-hour + less than 10 GB disk]. This validates C3 and the browser comparison in Section 6.

Preparation: Open `open-data/firefox-vs-chrome-plots.ipynb` with the archives in `open-data/outputs-firefox-vs-chrome/`.

Execution: Run all cells to load the paired Chrome and Firefox snapshots and regenerate the cookie-partitioning plots.

Results: The resulting figures show that Firefox forces third-party cookies into partitioned storage, while Chrome still exhibits widespread unpartitioned third-party cookies alongside CHIPS.

- (E3): **Run a new crawl** [30 person-minutes + ~36 compute-hours for the default top-10,000 setting with 20 parallel workers; shorter for smaller `-w` values. This validates the end-to-end measurement workflow behind C1.

Preparation: Complete the Docker setup above. Optionally choose a smaller website count for testing, e.g., `-w100`, and set `-p` to the desired GNU Parallel job count.

Execution: Run `bash analyze-ps-complete.sh -w 100 -p 4` for a small campaign or `bash analyze-ps-complete.sh` for the default top-10,000 campaign. The script downloads the Tranco list, extracts allowed domains, runs accept and deny crawls, checks contacted domains for attestation files, post-processes crawler outputs, and merges the accept/deny CSVs.

Results: A successful run ends with `Done!` and creates `outputs-YYYYMMDD.zip`. The archive contains `attested_domains.csv`, `allowed_domains.csv`, `allowed_attested.csv`, and `crawler_outputs.csv`, matching the schema documented in `open-data/outputs-longitudinal/README.md`.

A.5 Notes on Reusability

The provided analysis script’s behaviour can be further customized by changing parameters such as the browser used, its language, visit timeout, and number website visited, making it possible to run smaller functionality tests or scale the campaign to larger lists and multiple machines. The post-processing tools can also be modified to extract more or less information from the raw crawler outputs, allowing to analyse other crawling data such as the contents of local and session storage. Finally, we note that our results are longitudinal

and have been collected during a year-long crawling campaign. Therefore, we expect new crawls by evaluators to only validate the workflow and data format, not to exactly match our numerical results.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.